

ECE 315

Steven Knudsen, PhD, PEng

Lecture 20 :

- Planning peripherals and interfaces
- Design and build for test
- RTM



UNIVERSITY
OF ALBERTA

Last Lecture

- Parallel Interfacing
- Busses

This Lecture

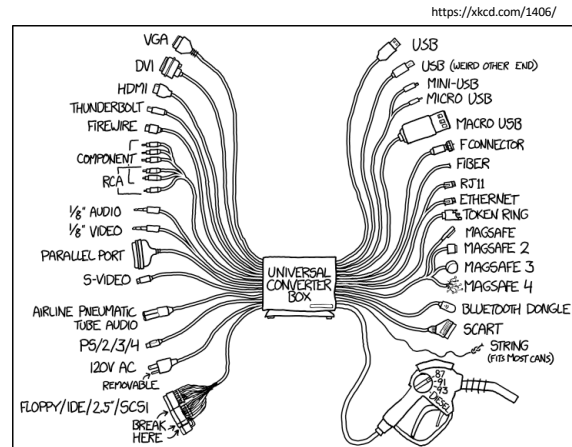
- Computer Interfaces, in general
- Planning peripherals and interfaces
- Design and build for test

Learning Objectives

- Characterize common computer interfaces
- Apply a structured design flow for embedded interface development
- Analyze and plan interface choices within real embedded systems
- Integrate design-for-test (DFT) techniques into embedded hardware and firmware
- Recognize the importance of manufacturing and certification considerations
- Know and apply best practices for validating embedded interfaces

Computer Interfaces, in general

- Computer interfaces are part of everyday life – ubiquitous
- Can categorize those we see everyday as
 - User Input Interfaces
 - Display & Output Interfaces
 - Wired (Physical) Computer Communication Ports
 - Wireless Communication Interfaces
 - Data & Peripheral Interfaces
 - Human-Machine Interfaces (HMI) in Appliances & Vehicles
 - Software Interfaces (UI/UX)



<https://xkcd.com/1406/>

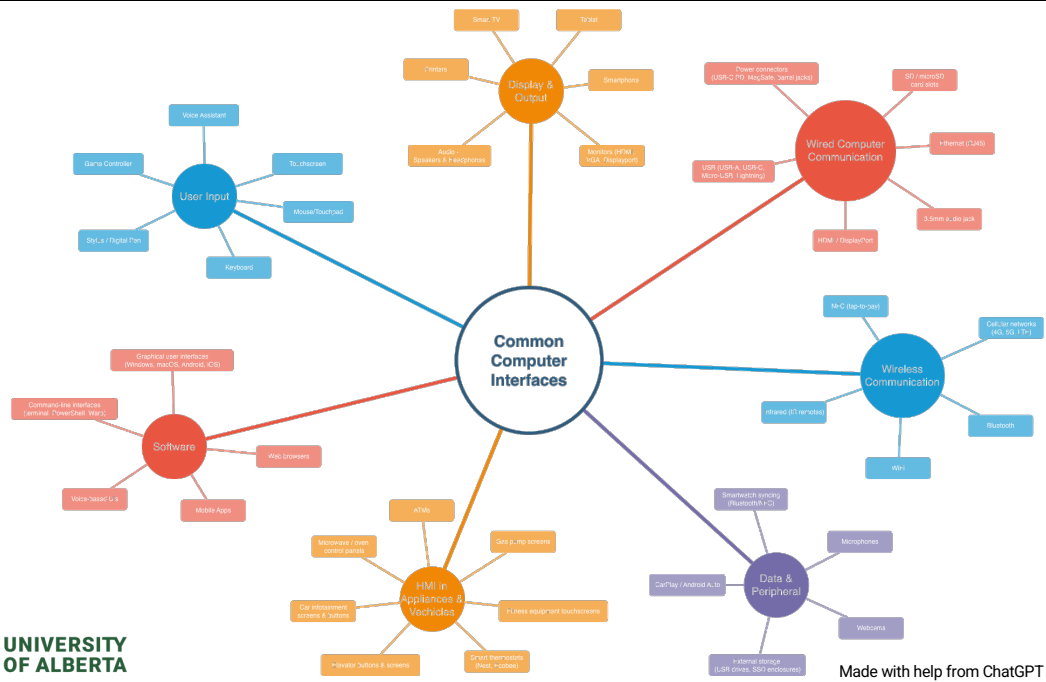
Made with help from ChatGPT

5

Computer Interfaces – Pre-configured, Plug'n'Play

The interfaces we encounter the most are generally either;

- Pre-configured
 - Automobile, appliance, entertainment and game system controls and displays
 - Voice and gesture
- Plug and Play
 - Computer peripherals, removeable storage,
 - Wireless – Bluetooth, WiFi
 - Web browsers, applications



Made with help from ChatGPT

Everyday Computer Interfaces

- Some, like Ethernet with protocols like Transmission Control Protocol/Internet Protocol (TCP/IP) and User Datagram Protocol (UDP) are covered in other courses
- Others, like USB, WiFi, Bluetooth are examples of a kind of protocol (serial)
- Arguably, they can be seen as variants on protocols used in embedded environments

Everyday Computer Interfaces

- Some, like Ethernet with protocols like Transmission Control Protocol/Internet Protocol (TCP/IP) and User Datagram Protocol (UDP) are covered in other courses
- Others, like USB, WiFi, Bluetooth are examples of a kind of protocol (serial)
- Arguably, they can be seen as variants on protocols used in embedded environments

We will focus here on embedded and be confident those concepts extend to higher-level protocols and physical interfaces

Design & Test of Embedded Computer Interfaces

Planning, Interface Design, and Design-for-Test Methods



UNIVERSITY
OF ALBERTA

10

General Design Flow

- Requirements definition
- What processor? – already selected or do we get to select?
- Selecting types of interfaces – serial or parallel, which standards
- Electrical considerations – low power, other signals, logic levels (3v3, 5v, 1v8?)
- Firmware driver design – design from scratch, use library, what does processor support
- Validation and design-for-test (DFT) integration
- Production test planning



Made with help from ChatGPT

11

Some Motivation

- Embedded systems must reliably communicate with sensors, actuators, and peripherals
- Interfaces often fail due to design, electrical, timing, or integration errors
- Early planning + test-friendly design reduces time-to-market and hardware iterations (board spins)
 - Hardware iterations are expensive and time-bound
- Software/firmware is easy and quick to change, update, but still costs

Design in General

- Designing almost anything in Engineering is an iterative process
 - The conceptual design phase for computer interfacing may not need iterating since the parts being interfaced to will have defined interfaces; (UART, SPI, I²C, parallel)
 - Proceed straight to block diagrams
-
- ```

graph TD
 Need[Need] --> Analysis[Analysis]
 Analysis --> SOP[Statement of Problem]
 SOP --> CD[Conceptual Design]
 CD --> SS[Selected Schemes]
 SS --> ES[Embodiment of Schemes]
 ES --> Detailing[Detailing]
 Detailing --> DP[Design Products]
 DP --> CD
 ES --> CD
 ES --> SOP
 ES --> Need
 CD --> AD1[Artefacts documents]
 SS --> AD2[Artefacts documents]

```
- UNIVERSITY OF ALBERTA
- Conceptual Design for Engineers, 3<sup>rd</sup> ed, Michael French
- 13



## Example – Toddler Monitor



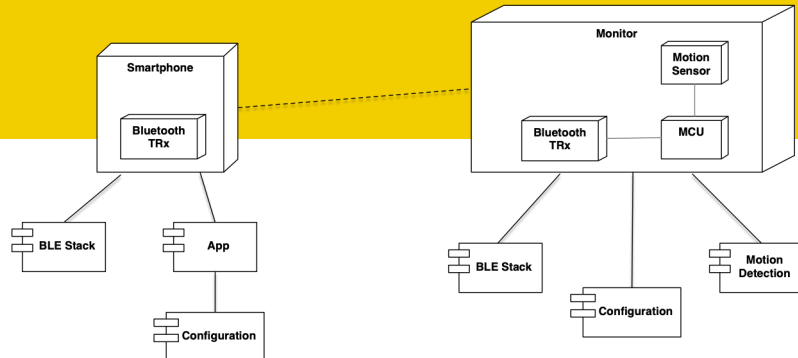
From ECE 492 – Steven Knudsen

14

This product, conceived of and developed by two mums in Calgary, is used to illustrate the development of an embedded system

## Block Diagram

- Use a deployment diagram to identify the main nodes and components
- The connections between nodes can be interfaces
- This diagram is for a commercial product, the Toddler Monitor



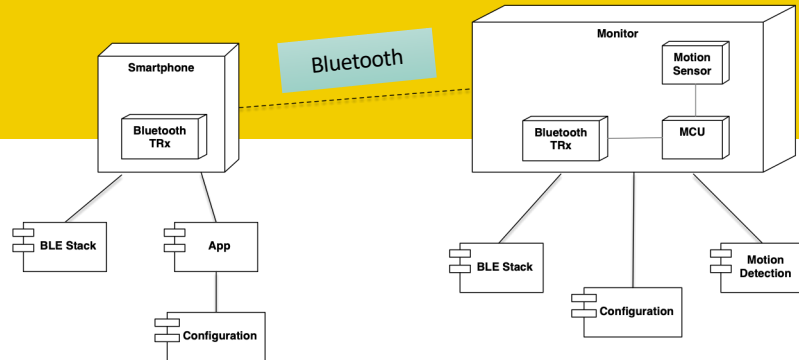
The Toddler Monitor helps keep toddlers from wandering around at night.

The monitor hangs on the toddler's bedroom door handle and if the door is opened while the toddler is supposed to be sleeping, the parent's smartphone alerts them.

We are not going into great detail with this example, just using to illustrate doing interface specification and design.

## Block Diagram

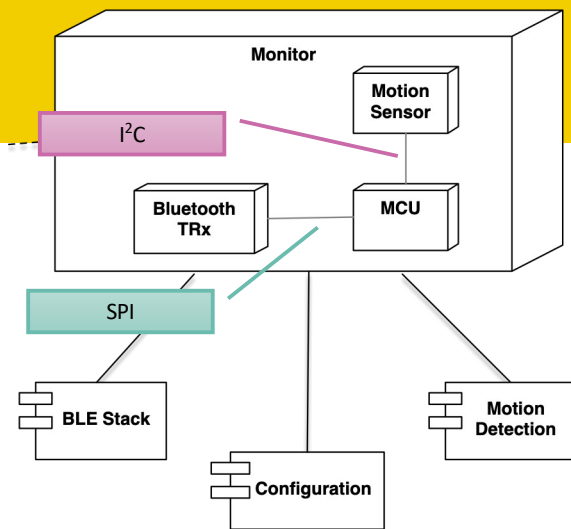
- Use a deployment diagram to identify the main nodes and components
- The connections between nodes can be interfaces
- Bluetooth is an obvious interface to the smartphone





## Block Diagram

- Use a deployment diagram to identify the main nodes and components
- The connections between nodes can be interfaces
- Within the Monitor there are two main interfaces
  - **I<sup>2</sup>C** – Data from motion sensor is relatively slow rate, so I<sup>2</sup>C is okay
  - **SPI** – Bluetooth can support up to 64 Mbps, so a faster interface is needed



# Design Flow

- Requirements definition
  - Detect motion – communicate with sensors
  - Communicate with smartphone – wireless communication
  - Battery powered – need long battery life and battery monitoring
- What processor?
  - Easiest is Bluetooth; since parents at home, its range is good enough
  - Lots of options – choose Nordic Semiconductor's nRF52xxx
    - ARM core, tried and proven, very low power consumption
  - Lots of vendors of pre-certified modules; advantage for initial production cost

# Design Flow

- Electrical considerations – low power, 3v3 for nRF52xxx
- Firmware driver design – Nordic libraries and examples, no license
- Validation and design-for-test (DFT) integration
  - Prototype and first version have test points, “blinky lights”, and programming interface
    - JTAG or Serial Wire Debug (SWD)
- Certifications
  - Bluetooth Special Interest Group (SIG) – fee to be paid
  - FCC, ISED, compliance with Part 15 rules for license-exempt operation
  - Start with pre-certified Bluetooth module for initial low volume run – reduces risk of time and cost



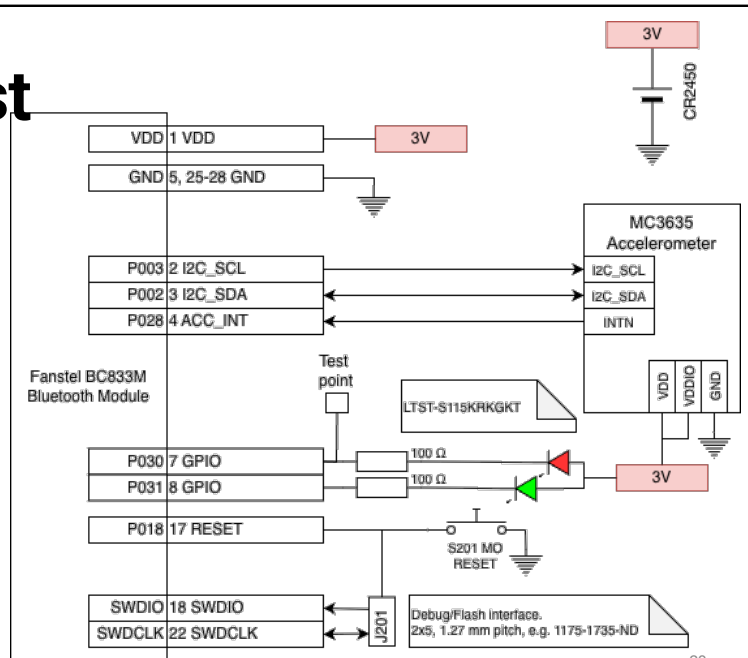
From ECE 492 – Steven Knudsen

19

SWD is really modified JTAG as done by ARM

# Design for Test

- Bluetooth and processor are integrated, so only need worry about I<sup>2</sup>C interface

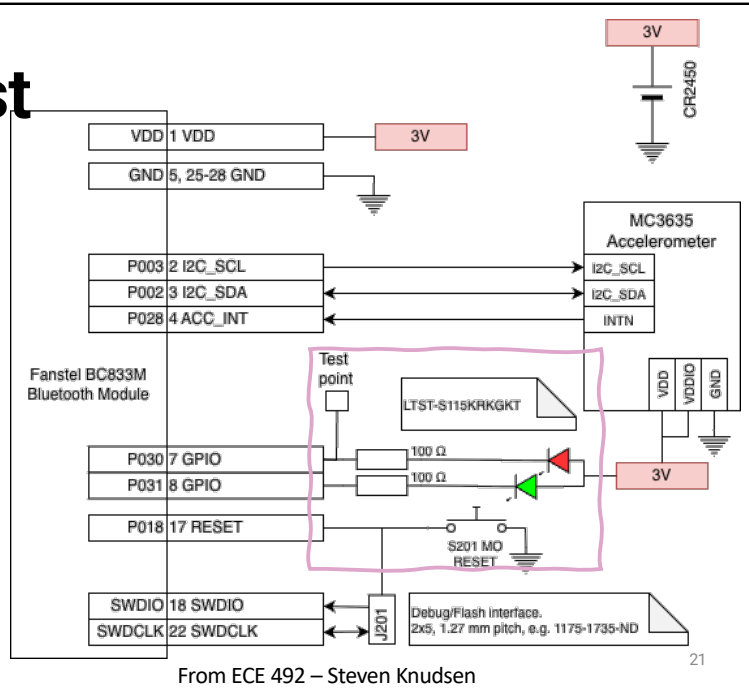


From ECE 492 – Steven Knudsen

20

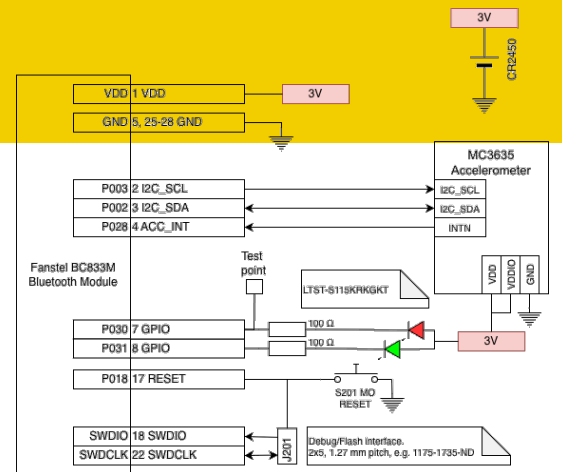
# Design for Test

- Bluetooth and processor are integrated, so only need worry about I<sup>2</sup>C interface
- The test point and LEDs are useful for debugging
- They and the reset switch don't need to be populated for the product



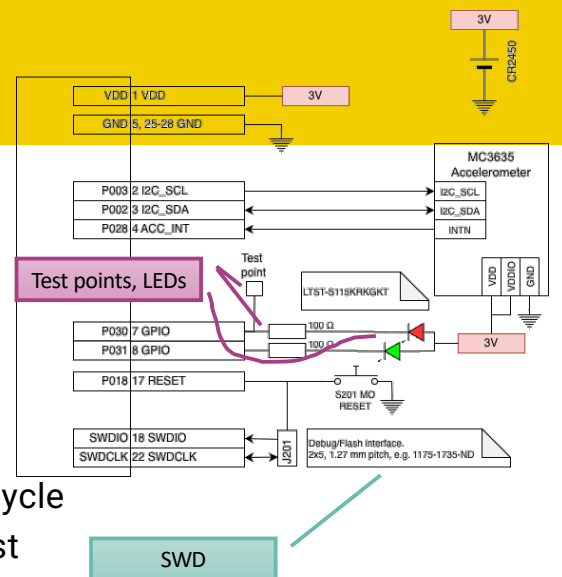
# Design for Test

- Faster prototyping
- Simplifies board bring-up
- Reduces manufacturing defects
- Enables automated testing
  - JTAG or SWD
  - Special test points
- Improves debugging throughout lifecycle



# Design for Test

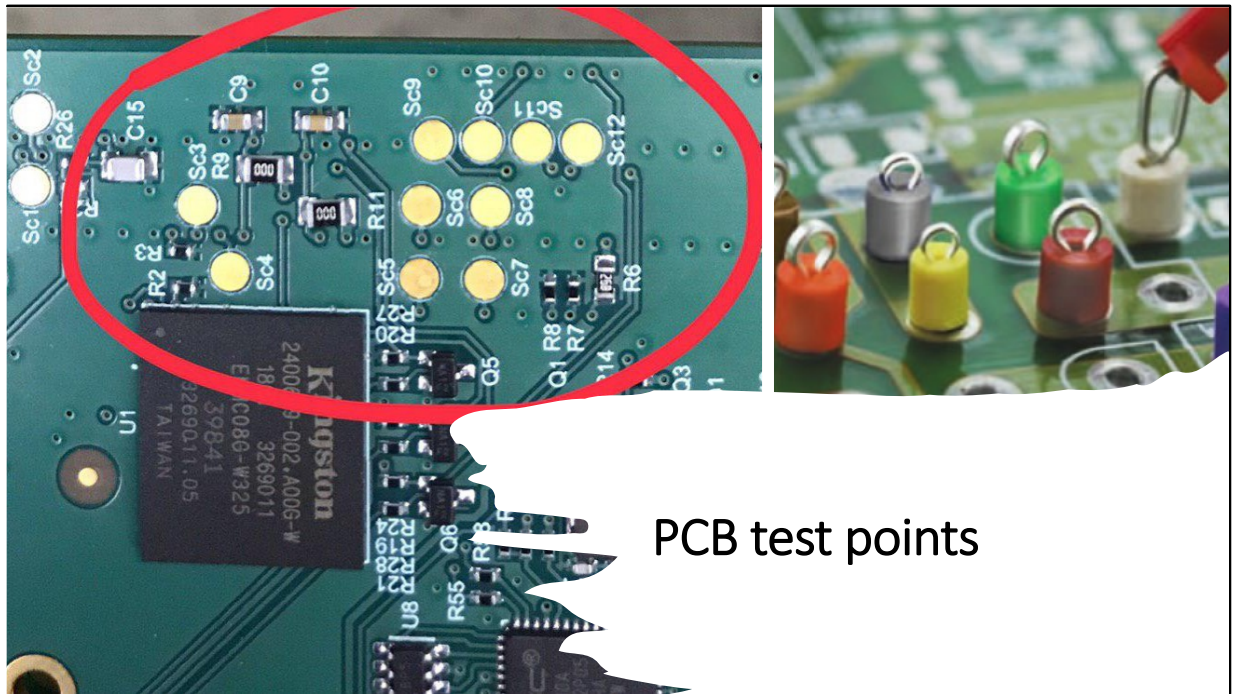
- Faster prototyping
- Simplifies board bring-up
- Reduces manufacturing defects
- Enables automated testing
  - JTAG or SWD
  - Special test points
- Improves debugging throughout lifecycle
- Don't populate for products, save cost



# Design for Test and Manufacture

- Hardware-level: test points, headers, boundary scan, built-in self-test (BIST)
- Firmware-level: debug logging, self-check routines
- System-level: loopback modes, functional tests
- Provide labeled test points on:
  - Power rails
  - Ground
  - Key digital buses (SCLK, MOSI, MISO, TX, RX)
  - Analogue signals before filtering
- Use pads accessible to pogo-pin fixtures
- Maintain spacing for probes





PCB test points



## PCB Test Ports and Headers

- JTAG/SWD (processor debugging)
- UART header for console
- SPI/I<sup>2</sup>C debug headers
- USB-C debug mode ports
- Vendor-specific programming ports
- Your own test points
- **Best practices**
  - Isolate via removable resistors or jumpers
  - Space components a little extra to make room for probes

## Joint Test Action Group (JTAG)

- An industry standard for verifying designs of and testing printed circuit boards after manufacture
- Often built into chips
- Used as the primary means of accessing sub-blocks of integrated circuits, *otherwise they are not accessible*
- Used to program internal flash, such as found on Zynq and RP2040
- Can verify connectivity of chips, critical for ball-grid array (BGA) devices that can have hundreds of connections
- **Best Practice:**
  - Have JTAG connector for prototypes, keep pads for production just in case



Wikipedia contributors. "JTAG." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 29 Oct. 2025. Web. 23 Nov. 2025.

28

# Built-in Self-Test (BIST)

- Typical approaches:
  - Memory self-test
  - Peripheral loopback (internal/external)
  - Analogue self-test (comparator checks, ADC ramp tests)
  - Pattern generation for I/O pins
    - This is where the test point pads and use of pogo-pins or flying probes comes in
  - States and results are logged to FLASH or output via UART, or both
- **Best Practices:**
  - Like JTAG, have UART header that is optional to populate
  - Use LEDs to signal status (e.g., use combination of state and patterns)

# Certifications

- We see everyday marks on products like UL, CE, FCC
- Various organizations exist to regulate products and systems for safety and commercial reasons
- Responsibility of the designer and engineers to ensure that products and systems are compliant
- This always costs in time and money; wise to think of from the start
- Plan for compliance pre- and final-testing, again, time and cost
  - Pre-testing saves money and time, de-risks designs



## Computer Interfaces – Best Practices Summary

- Plan interface requirements early
- Integrate DFT features at PCB and firmware levels
- Always include test points, headers, and ground access
- Add self-test routines
- Validate interfaces using both hardware and software tools
- Maintain documentation and schematics updated for test teams

## Next Lecture

- RTM
- Memory and memory-mapped I/O



# Questions

